D2L代码速成八讲

（PyTorch版）

以李沐《动手学深度学习》(D2L)的代码为主线

助你看懂D2L所有代码

先学速成八讲看懂代码意思，再学D2L看懂代码逻辑

注解

先认人、书、缩写和工具，后面的代码就更容易读。

李沐

《动手学深度学习》的作者之一、同名课程的主讲人，长期从事深度学习研究与工程实践。
书中“先讲清为什么，再给可运行代码”的写法，来自他和合著者的教学经验。

《动手学深度学习》

李沐等人编写的开源教材，英文名 Dive into Deep Learning。内容按“数学推导 + 可运行代码 + 讲解”组织，中文版可在 zh.d2l.ai 免费阅读。

D2L

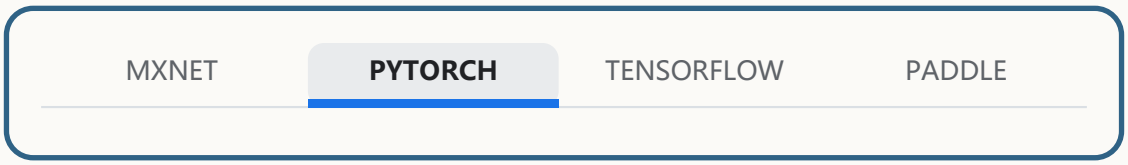
《动手学深度学习》的英文名是 Dive into Deep Learning，D2L 即其缩写：既指这本教材，也指随书代码库里的 d2l 工具包。书中的 from d2l import torch as d2l 导入的就是它，其中封装了数据加载、训练循环、绘图等教学辅助函数。

PyTorch

Meta 开源的深度学习框架，以张量运算和动态计算图为核心。本书按 D2L 的 PyTorch 实现读代码；同一本教材另有 MXNet、TensorFlow 等版本。

本书所说的“代码”，均指 D2L 中文版 PyTorch 实现中的示例代码。

> 学习D2L建议使用官网文字版教材，而非视频课。
>
> D2L官网：https://zh-v2.d2l.ai/



学习D2L时，注意选择代码版本为“**PYTORCH**”。

目录

第一章 代码全局结构 1

1.1 数据准备与整理 2
1.2 模型构建 2
1.3 损失与参数学习 3
1.4 训练流程概览 4

第二章 张量基础操作 5

2.1 创建张量 5
2.2 形状、轴数与元素总数 5
2.3 数据类型与转换 6
2.4 索引、切片与赋值 7
2.5 改变形状 8
2.6 增减长度为 1 的轴 8
2.7 调整轴顺序 9
2.8 提取 Python 数字 10
2.9 综合示例：读取与修改张量 10

第三章 张量运算 11

3.1 逐元素运算 11
3.2 求和、均值与轴 11
3.3 保留汇总轴 12
3.4 广播规则 13
3.5 拼接与堆叠 14
3.6 点积与矩阵乘法 15
3.7 比较、筛选与计数 15
3.8 综合示例：softmax 16

第四章 数据加载与预处理 18

4.1 数据集与数据加载器 18
4.2 解析 load_array 19
4.3 数据加载参数 20
4.4 批次解包 20
4.5 迭代器与遍历控制 21
4.6 生成器与 yield 22
4.7 图像预处理 23
4.8 表格预处理 24
4.9 常见 Python 语法 24
4.10 综合示例：检查图像批次 25

第五章 模型定义与参数 26
5.1 创建模型与调用模型 26
5.2 顺序容器 Sequential 26
5.3 全连接层的参数与计算 27
5.4 自定义模型 Module 28
5.5 模型调用与函数式接口 29
5.6 卷积、激活与池化 29
5.7 综合示例：LeNet 30
5.8 参数访问与管理 31
5.9 参数初始化 32
5.10 逐层检查输出形状 33
第六章 自动求导、损失与优化器 34
6.1 开启梯度追踪 34
6.2 反向传播与参数梯度 35
6.3 单元素目标与损失汇总 35
6.4 梯度清理 36
6.5 分离结果与关闭梯度记录 37
6.6 损失函数与损失值 37
6.7 损失汇总参数 reduction 38
6.8 交叉熵的输入要求 39
6.9 优化器的职责 39
6.10 手写 SGD 与梯度缩放 40
6.11 综合示例：单批参数更新 41
第七章 训练与评估 42
7.1 训练模式与评估模式 42
7.2 轮次、批次与更新步 43
7.3 预测类别与 argmax 43
7.4 正确数量与准确率 44
7.5 指标累加器 45
7.6 解析单轮训练函数 46
7.7 整轮指标的加权统计 47
7.8 解析评估函数 47
7.9 多轮训练与指标组合 48
7.10 结果展示与检查 49
第八章 设备、存取与综合阅读 50
8.1 计算设备与设备选择 50
8.2 模型与数据的设备迁移 51
8.3 张量与容器的存取 51
8.4 模型状态字典 52
8.5 重建模型与加载状态 52
8.6 加载设备映射与一致性检查 53
8.7 训练检查点 54
8.8 D2L 辅助函数 55
8.9 综合阅读：多层感知机训练 56

第一章 代码全局结构

本书适合会基础 Python、刚接触 PyTorch 的读者。通过李沐编写的《动手学深度学习》课程（D2L）的常见代码，认识每种写法的作用，进而读懂数据处理、模型定义和训练循环。

阅读代码时，抓住四件事：数据从哪里来，模型怎样计算，参数怎样学习，结果怎样评估和保存。先看懂每段的作用，再理解它们为什么这样连接。

全书结构与阅读方式

章节	核心内容	阅读后应能解释
第一章	代码全局结构	数据、模型、损失与优化器怎样配合
第二章	张量基础操作	创建、索引、变形和类型转换怎样影响数据
第三章	张量运算	轴、广播、汇总和矩阵乘法怎样决定结果
第四章	数据加载与预处理	单条样本怎样组织成模型接收的一批数据
第五章	模型定义与参数	网络怎样连接，参数在哪里，形状怎样变化
第六章	自动求导、损失与优化器	损失、梯度与参数更新之间的关系
第七章	训练与评估	多轮训练、模式切换和指标统计的逻辑
第八章	设备、存取与综合阅读	GPU 迁移、状态恢复及 D2L 封装的作用

每章按“用途 → 代码 → 数据与形状”展开。初读时抓住常用写法；系统学习 D2L 时，再回查具体细节。

示例与记号约定

示例以 D2L 中文版的 PyTorch 实现为依据，相关章节名称附有链接。中文注释用于说明语句作用；合并单元、删节及教学补充均在上下文标明。代码片段可能依赖前文定义的数据或函数，不能把所有片段依次粘贴成一份程序。

`B` 表示当前批的样本数，`C` 表示通道数或类别数，具体含义由上下文说明。图像张量通常写成 `(B, C, H, W)`，依次对应批次、通道、高和宽。`(4,)` 是一维张量的形状，`()` 是标量形状。轴从 0 编号，形状本身不包含各轴的实际含义。

教材中的输出通常来自交互式环境：单独写 `X.shape` 可以显示结果；在普通 Python 脚本中，需要使用 `print(X.shape)`。不同版本的 PyTorch、torchvision、pandas 和 d2l 可能存在接口差异；出现兼容问题时，应核对安装版本与对应文档。

1.1 数据准备与整理

下列写法选自 D2L 的 2.1 数据操作和 3.3 线性回归简洁实现。这些是分散出现的片段，不是一份连续程序。

代码写法	作用与含义
`import torch`	导入 PyTorch
`from torch import nn`	导入神经网络工具，后面用 `nn.xxx`
`from d2l import torch as d2l`	导入教材提供的 PyTorch 辅助工具，简称 `d2l`
`from torch.utils import data`	导入 PyTorch 的数据工具，后面用 `data.DataLoader` 等接口
`x = torch.arange(12)`	生成从 0 到 11 的张量
`x.shape`	查看每个维度的大小
`x.numel()`	查看总共有多少个元素
`X = x.reshape(3, 4)`	把这些元素整理成 3 行 4 列
`torch.cat((X, Y), dim=0)`	沿第 0 条轴拼接两个张量；二维时相当于上下拼接
`torch.cat((X, Y), dim=1)`	沿第 1 条轴拼接；二维时相当于左右拼接
`data.TensorDataset(*data_arrays)`	把几组对应的数据包装成数据集，例如特征和标签
`data.DataLoader(dataset, batch_size, shuffle=is_train)`	每次取一批数据，是否打乱由 `is_train` 决定
`next(iter(data_iter))`	从数据加载器中取出第一批，常用来检查数据

`shape` 描述各轴长度，`numel()` 给出元素总数，`reshape` 在元素总数不变的前提下改变形状。三者分别回答“怎样排列”“一共有多少”和“改成什么排列”。

导入来源决定了接口属于哪个工具库：

- `torch.xxx`：PyTorch 提供的功能。
- `d2l.xxx`：教材为了减少重复代码而封装的功能，内部可能包含很多步骤。
- `data.xxx`：PyTorch 的数据加载工具，即 `torch.utils.data`。

1.2 模型构建

选自 D2L 的 4.3 多层感知机和 5.1 层和块。

代码写法	作用与含义
`nn.Linear(784, 256)`	创建一个全连接层，把每个样本的 784 个特征转换成 256 个特征
`nn.ReLU()`	创建一个 ReLU 激活层
`nn.Flatten()`	默认保留批次维，把后面的维度展平成一维特征

代码写法	作用与含义
`nn.Sequential(...)`	把里面的层按顺序连接起来
`class MLP(nn.Module):`	定义一个名叫 `MLP` 的模型类，继承 PyTorch 的模型基础功能
`super().__init__()`	先完成父类的初始化，让模型能正常管理层和参数
`self.hidden = nn.Linear(20, 256)`	创建并登记隐藏层，属性名为 `hidden`
`def forward(self, X):`	规定输入 `X` 进入模型后，要经过什么计算
`net(X)`	把 `X` 送入模型，执行前向计算，得到输出
`net.parameters()`	获取模型参数，常交给优化器去调整

D2L 4.3 的模型定义如下，补充了中文注释：

```
net = nn.Sequential(
    nn.Flatten(),          # 把每张图片展开成特征向量
    nn.Linear(784, 256),   # 784 个特征转换成 256 个
    nn.ReLU(),            # 做非线性激活
    nn.Linear(256, 10)     # 转换成 10 个类别的分数
)
```

这一模型把图像转换为类别分数。各层的具体作用将在第五章展开；此处关注数据按什么顺序经过这些层。

输入数据经过模型得到输出，这个过程叫“前向计算”。模型中的权重和偏置是需要训练的参数；批大小、学习率和训练轮数由程序设置，通常称为“超参数”。

1.3 损失与参数学习

选自 D2L 的 2.5 自动微分和 3.3 线性回归简洁实现。

代码写法	作用与含义
`x.requires_grad_(True)`	开始追踪涉及 `x` 的计算，准备对它求梯度
`x.grad`	查看已经算出的、关于 `x` 的梯度
`nn.MSELoss()`	创建均方误差损失函数，常用于回归
`loss(net(X), y)`	让模型预测，再拿预测结果和正确答案 `y` 计算损失
`torch.optim.SGD(net.parameters(), lr=lr)`	创建 SGD 优化器，用指定学习率调整模型参数
`trainer.zero_grad()`	清除上一次留下的梯度
`l.backward()`	从损失 `l` 出发，计算相关参数的梯度
代码写法	**作用与含义**
`trainer.step()`	根据梯度更新参数

代码写法	作用与含义
`with torch.no_grad():`	下面缩进范围内的计算不记录求梯度所需的信息

训练代码中最基本的职责划分是：

`backward()` **负责算梯度；** `step()` **负责改参数。**

D2L 常用 `trainer` 命名优化器，其他程序也可能使用 `optimizer`。变量的职责由创建方式和调用位置决定。

1.4 训练流程概览

下面是 D2L **3.3 节的 PyTorch 训练循环**，只添加了中文注释；假设前面的数据、模型、损失函数和优化器已经准备好。

```
num_epochs = 3                          # 完整训练 3 轮

for epoch in range(num_epochs):          # 外层：控制训练轮次
    for X, y in data_iter:               # 内层：依次取出一批特征和标签
        l = loss(net(X), y)              # 预测，然后计算这一批的损失
        trainer.zero_grad()              # 清除旧梯度
        l.backward()                     # 计算新梯度
        trainer.step()                   # 更新模型参数

    l = loss(net(features), labels)      # 每轮结束后计算全部训练数据的损失
    print(f'epoch {epoch + 1}, loss {l:f}')  # 显示轮次和损失
```

这段循环的整体逻辑是：

总共训练三轮。每一轮不断取出一批数据，预测、计算损失、清除旧梯度、计算新梯度、更新参数。全部批次处理完，再查看这一轮的训练损失。

注意最后两行的缩进：它们在内层循环外，所以**每轮执行一次。**

第二章 张量基础操作

张量可以先理解为能参与计算的多维数字数组。一个数、一排数、一张数表，都可以用张量表示。本章重点区分三件事：里面存了什么数、这些数怎样排列、用什么类型存储。

本章以 D2L 2.1 数据操作为主线。表格中的简化示例用于比较相近操作，原文片段和教学补充在相应位置说明。

2.1 创建张量

常见写法	作用与含义	注意
`torch.tensor([1, 2, 3])`	根据给定数据创建张量	内容由传入数据决定
`torch.tensor([[1, 2], [3, 4]])`	创建一个二维张量	形状是 `(2, 2)`
`torch.arange(12)`	生成 0 到 11	不包含结束值 12
`torch.zeros((2, 3, 4))`	创建形状为 `(2, 3, 4)` 的全零张量	括号里的数是各轴长度
`torch.ones((2, 3, 4))`	创建同样形状的全一张量	所有元素都是 1
`torch.randn(3, 4)`	创建 3 行 4 列的标准正态随机数	不是 0 到 1 的均匀随机数

创建张量时，数据参数与形状参数需要区分：

```
torch.tensor([2, 3])     # 内容是 2 和 3，形状为 (2,)
torch.zeros((2, 3))      # 内容全是 0，形状为 (2, 3)
```

`torch.tensor` 根据给定数据创建张量；`torch.zeros` 根据指定形状创建全零张量。

随机数张量常用于构造示例输入、检查模型输出形状或初始化参数。`randn` 生成标准正态随机数，不能与生成均匀随机数的 `rand` 混淆。

2.2 形状、轴数与元素总数

以下面的张量作为例子：

```
X = torch.arange(12).reshape(3, 4)
```

它的内容是：

```
[[ 0,  1,  2,  3],
 [ 4,  5,  6,  7],
 [ 8,  9, 10, 11]]
```

写法	意思	对这个 `X` 的结果
`X.shape`	每条轴分别多长	`torch.Size([3, 4])`
`X.shape[0]`	第 0 条轴的长度	`3`
`X.shape[1]`	第 1 条轴的长度	`4`
`X.ndim`	一共有几条轴	`2`

写法	意思	对这个 X 的结果
X.numel()	总共有多少个元素	12
X.size()	查看形状，作用类似 shape	torch.Size([3, 4])
X.size(0)	查看第 0 条轴的长度	3
len(X)	第一条轴的长度	3

以后遇到：

```
batch_size = X.shape[0]
```

在这个示例约定下，X.shape[0] 就是批大小。

但前提是这段程序确实把第 0 条轴用作样本轴。**轴的实际含义来自代码约定，不能只靠数字猜。**

size() **返回形状，** numel() **返回元素总数。** 两者不能互换。PyTorch 张量参考

2.3 数据类型与转换

D2L 2.1 中有这条真实语句：

```
X = torch.arange(12, dtype=torch.float32).reshape((3, 4))
```

下面是常见补充写法：

写法	作用与含义
X.dtype	查看数据类型
X.float()	转为 float32
X.long()	转为 int64
X.to(dtype=torch.float32)	明确指定转成 float32
X.type_as(Y)	转成与 Y 相同的张量类型

数据类型应与任务和接口的要求对应：

- 模型的连续输入、权重：经常用浮点数。
- 分类任务的类别编号：经常用整数。
- .float() **不会把数值自动归一化到 0～1**，它只是类型转换。

类型转换通常需要使用返回值：

```
X = X.float()          # 后续 X 使用转换后的结果
```

类型不匹配会导致计算失败。例如，使用类别编号作为交叉熵标签时，通常需要 torch.long 类型。第六章将结合损失函数说明输入和标签的要求。

返回值与原地修改

`X = Y` 让两个变量指向同一个对象，并不复制数据。需要独立数据时可使用 `Y.clone()`。切片或变形得到的张量也可能与原张量共享存储，因此“得到了一个返回值”并不保证两份数据互不影响。阅读赋值时，应同时检查变量指向谁，以及数据是否被修改。

2.4 索引、切片与赋值

继续使用形状为 `(3, 4)` 的 `X`。以下各行都独立理解，不修改它。

写法	作用与含义	结果形状
`X[0]`	取第 1 行	`(4,)`
`X[-1]`	取最后一行	`(4,)`
`X[1:3]`	取索引 1、2 两行	`(2, 4)`
`X[:, 0]`	所有行，取第 1 列	`(3,)`
`X[:, 1:3]`	所有行，取索引 1、2 两列	`(3, 2)`
`X[1, 2]`	取索引为“行 1、列 2”的单个数	`()`
`X[:2, :]`	前两行，所有列	`(2, 4)`
`X[::2]`	行索引每隔 2 取一次，即第 0、2 行	`(2, 4)`

索引表达式遵循以下规则：

规则一：逗号分隔不同的轴。

```
X[行的选择方式，列的选择方式]
```

规则二：冒号表示范围，左边包含、右边不包含。

```
1:3       # 取索引 1、2
:2        # 从开头取到索引 2 之前
:         # 全部
::2       # 从开头开始，步长为 2
```

整数索引与切片对形状的影响不同：

```
X[0]      # 形状 (4,)：用整数索引，取走这一轴
X[0:1]    # 形状 (1, 4)：用切片，保留这一轴
```

它们取出的数字相同，但**形状不同。**保留样本轴时，这个区别很重要。

D2L 原文还使用：

```
X[1, 2] = 9             # 修改第 2 行、第 3 列的元素
X[0:2, :] = 12          # 把前两行的所有元素改为 12
```

索引表达式位于等号左侧时，表示向选中位置写入数据。整数索引越界会报错；切片超出边界通常会截到实际范围。D2L 索引和切片

2.5 改变形状

用途：把已有数据整理成下一步计算所需的排列方式。

写法	作用与含义
`x.reshape(3, 4)`	整理成 3 行 4 列
`x.reshape((3, 4))`	同样的意思，形状作为一个元组传入
`x.reshape(-1, 4)`	每行 4 个，行数自动计算
`x.reshape(3, -1)`	固定 3 行，列数自动计算
`X.reshape(-1)`	展开成一长排
`X.flatten()`	默认把所有轴展开成一条轴
`X.view(3, 4)`	也按指定形状查看数据，但对内存布局有要求

D2L 2.1 用 `reshape` 演示变形；`flatten`、`view` 是这里补充认识的同类写法。

这里的 `-1` 表示“根据元素总数自动算”，最多只能有一个。

```
x = torch.arange(12)

x.reshape(-1, 4)        # 12 ÷ 4，得到 (3, 4)
x.reshape(2, -1)        # 12 ÷ 2，得到 (2, 6)
```

形状变换的限制如下：

- `reshape` 不会自动补足或丢弃元素，目标形状的元素数必须一致。
- `X.reshape(...)` 通常不会修改 `X` 自身的形状，需要使用返回值。
- `view` 要求形状与原有数据排列兼容；交换轴后有时无法直接使用。`reshape` 在需要时可以复制数据，但不能假定它总会复制。

保留批次轴、只展平样本内部维度时，可使用：

```
X.flatten(start_dim=1)
```

例如：

```
(32, 1, 28, 28) → (32, 784)
```

这就是常见的“保留 32 个样本，把每张图片展开”。

2.6 增减长度为 1 的轴

以下为教学补充，用于调整接口要求的形状。插入或删除长度为 1 的轴，不改变元素总数。

假设：

```
x = torch.arange(4)     # 形状 (4,)
```

写法	结果形状	作用与含义
`x.unsqueeze(0)`	`(1, 4)`	在最前面插入一条长度为 1 的轴
`x.unsqueeze(1)`	`(4, 1)`	在位置 1 插入一条长度为 1 的轴
`x.unsqueeze(-1)`	`(4, 1)`	在最后插入一条长度为 1 的轴

常见应用：

```
一张图片：(3, 224, 224)
        ↓ unsqueeze(0)
一批图片：(1, 3, 224, 224)
```

模型可能要求输入包含批次轴。即使只有一张图片，也需要明确表示“这一批有 1 张”。PyTorch unsqueeze

反过来：

```
y = x.unsqueeze(0)      # (1, 4)

y.squeeze(0)            # (4,)
y.squeeze()             # 删除所有长度为 1 的轴
```

`squeeze` **只能删除长度为 1 的轴。** 对 `(3, 4)` 使用 `squeeze()`，不会变成一维。

`squeeze` 删除的是长度为 1 的轴。不指定轴时，批大小恰好为 1 的批次轴也可能被删除；需要保留批次结构时，应显式指定要删除的轴。PyTorch squeeze

2.7 调整轴顺序

D2L 2.3 线性代数使用：

```
A.T                     # 对二维矩阵做转置，交换行和列
```

交换轴还可以使用以下接口：

写法	含义
`X.transpose(0, 1)`	交换第 0 条轴和第 1 条轴
`X.permute(0, 3, 1, 2)`	把原来的第 0、3、1、2 条轴，依次作为新的各条轴

例如：

```
原形状：(32, 224, 224, 3)
轴编号：  0    1    2  3

permute(0, 3, 1, 2)

新形状：(32, 3, 224, 224)
```

`permute` **调整轴的顺序；** `reshape` **重新组织元素的形状。图片通道换位置时，不能只靠把目标形状写进** `reshape`**。** PyTorch permute

2.8 提取 Python 数字

D2L 2.1 的真实片段：

```
a = torch.tensor([3.5])
a, a.item(), float(a), int(a)
```

表达式	得到什么
`a`	张量 `tensor([3.5000])`
`a.item()`	普通 Python 数字 `3.5`
`float(a)`	Python 浮点数 `3.5`
`int(a)`	Python 整数 `3`，这里不是四舍五入

`.item()` 常见于打印或记录损失：

```
loss_value = loss.item()
```

`.item()` 仅适用于单元素张量。多元素张量调用它会报错；得到的 Python 数字不再保留原张量的求导关系。

2.9 综合示例：读取与修改张量

下面把 D2L 2.1 中分散出现的语句组合成一个教学片段，添加中文注释。注意最后两行会修改数据。

```
import torch

# 生成浮点数，整理成 3 行 4 列
X = torch.arange(12, dtype=torch.float32).reshape((3, 4))

# 查看最后一行，以及第 2、3 行
X[-1], X[1:3]

# 修改第 2 行、第 3 列
X[1, 2] = 9

# 把前两行全部改为 12
X[0:2, :] = 12
```

最终内容应当是：

```
[[12., 12., 12., 12.],
 [12., 12., 12., 12.],
 [ 8.,  9., 10., 11.]]
```

对单个元素写入的 9 被最后一次赋值覆盖。连续代码的结果取决于执行顺序，应区分读取操作、返回新结果的操作和原地修改操作。

第三章 张量运算

张量怎样计算，既要看数值，也要看形状。本章介绍对应位置计算、按轴求和、广播、拼接和矩阵乘法，并说明怎样判断结果形状。

本章结合 D2L 2.1 数据操作、2.3 线性代数与 3.6 的 softmax 实现。教学变式用于演示形状规则。

3.1 逐元素运算

D2L 2.1 的真实片段，添加中文注释：

```
x = torch.tensor([1.0, 2, 4, 8])
y = torch.tensor([2, 2, 2, 2])

x + y, x - y, x * y, x / y, x**y
```

这里 x、y 都是形状 (4,) 的张量。

写法	作用与含义	上例结果
x + y	对应位置相加	[3, 4, 6, 10]
x - y	对应位置相减	[-1, 0, 2, 6]
x * y	对应位置相乘	[2, 4, 8, 16]
x / y	对应位置相除	[0.5, 1, 2, 4]
x ** y	对应位置做乘方	[1, 4, 16, 64]
x ** 2	每个元素平方	[1, 4, 16, 64]
x + 1	每个元素加 1	[2, 3, 5, 9]

`*` 表示逐元素乘法；`@`、`torch.mm()` 等接口执行矩阵乘法。 两类运算具有不同的计算和形状规则。

以下函数也对每个元素分别计算：

写法	作用与含义
torch.exp(x)	每个元素分别计算 e^x
torch.log(x)	每个元素分别取自然对数
torch.abs(x)	每个元素分别取绝对值
torch.sqrt(x)	每个元素分别开平方

通常它们保留输入形状。注意定义域：实数张量中，负数开平方可能得到 nan，log(0) 会得到负无穷，并不一定直接抛出异常。

3.2 求和、均值与轴

用途：统计总量、求平均损失、按样本或按特征汇总。

下面用一个教学变式说明：

```
A = torch.tensor([
    [1.0, 2, 3],
    [4.0, 5, 6]
])
# 形状：(2, 3)
```

写法	含义	结果
A.sum()	所有元素相加	21
A.mean()	所有元素取平均	3.5
A.sum(dim=0)	沿第 0 条轴求和	[5, 7, 9]
A.sum(dim=1)	沿第 1 条轴求和	[6, 15]
A.mean(dim=0)	每列取平均	[2.5, 3.5, 4.5]
A.mean(dim=1)	每行取平均	[2, 5]

对于求和、求平均这类汇总运算，dim 指定被汇总的轴。 未设置 keepdim=True 时，这条轴不再出现在结果形状中。

```
A 的形状：(2, 3)
           ↑  ↑
          dim0 dim1

sum(dim=0)：汇总掉第 0 条轴 → 剩下 (3,)
sum(dim=1)：汇总掉第 1 条轴 → 剩下 (2,)
```

因此，二维表格中：

- dim=0：跨行汇总，得到每列的统计值。
- dim=1：跨列汇总，得到每行的统计值。

“跨行”与“跨列”只适用于这里的二维示例。对于任意维数，都应依据轴编号和形状判断汇总方向。

D2L 2.3 也出现了：

```
A.sum(axis=0)
```

这段教材使用 axis 指定轴；对应的常用 PyTorch 写法是 dim=0。

mean() 要求浮点或复数输入。整数张量应先转换类型：

```
A.float().mean()          # 先转浮点，再求平均
```

假设 losses 的形状是 (32,)，losses.mean() 就是“把 32 个损失汇总成一个平均损失”。

3.3 保留汇总轴

用途：让汇总结果保持便于后续运算的形状。

继续使用上面的 A：

```
A.sum(dim=1)                 # 形状：(2,)
A.sum(dim=1, keepdim=True)   # 形状：(2, 1)
```

内容分别可以理解为：

```
不保留轴：  [6, 15]

保留轴：   [[ 6],
          [15]]
```

写法	形状
A.sum(dim=0)	(3,)
A.sum(dim=0, keepdim=True)	(1, 3)
A.sum(dim=1)	(2,)
A.sum(dim=1, keepdim=True)	(2, 1)

keepdim=True **没有保留原来所有的数据，只是让被汇总的轴留下，长度变成 1。** PyTorch 求和参数

D2L 部分片段使用 keepdims=True。阅读 PyTorch 代码时应同时认识常用参数名 keepdim=True，编写代码时以安装版本支持的签名为准。

3.4 广播规则

用途：给每个样本加同一组偏置、让每一行除以自己的总和等。

D2L 2.1 的真实例子：

```
a = torch.arange(3).reshape((3, 1))  # 形状：(3, 1)
b = torch.arange(2).reshape((1, 2))  # 形状：(1, 2)

a + b
```

可以把计算过程想象成：

```
a:        b:

[[0],     [[0, 1]]
 [1],
 [2]]

结果：
[[0, 1],
 [1, 2],
 [2, 3]]
```

a 的一列与 b 的一行按广播规则配合，结果形状为 (3, 2)。不需要先手动复制它们。

广播是否成立，可依据以下规则检查：

从右向左对齐形状；对应位置要么相等，要么其中一个是 1；缺少的轴按 1 看。

两个形状	能否广播	结果形状
(3, 4) 与 (4,)	能	(3, 4)
(3, 4) 与 (3, 1)	能	(3, 4)
(3, 1) 与 (1, 4)	能	(3, 4)
(3, 4) 与 (3,)	不能	最后一轴的 4 和 3 不匹配

一个很容易忽略的陷阱：

```
预测值：(32, 1)
标签：  (32,)
相减后：(32, 32)
```

代码可能能运行，但已经不是“32 个预测分别减去对应标签”。 所以读损失代码时，必须留意预测与标签的形状。

按行归一化体现了保留轴与广播之间的配合：

```
# 教学变式：让每行除以自己的总和
row_sums = A.sum(dim=1, keepdim=True)  # (2, 1)
result = A / row_sums                  # (2, 3)
```

keepdim=True 留下的那条轴，使每行的总和能广播到该行的每个元素。

对于 (batch, features) 加 (features,)，表示“给每个样本加同一组数”。

3.5 拼接与堆叠

用途：把多份数据或特征合起来。

D2L 2.1 的真实片段：

```
X = torch.arange(12, dtype=torch.float32).reshape((3, 4))
Y = torch.tensor([[2.0, 1, 4, 3],
                  [1, 2, 3, 4],
                  [4, 3, 2, 1]])

torch.cat((X, Y), dim=0), torch.cat((X, Y), dim=1)
```

两者都是 (3, 4)：

写法	作用与含义	结果形状
torch.cat((X, Y), dim=0)	沿第 0 条轴接起来，二维时是上下接	(6, 4)
torch.cat((X, Y), dim=1)	沿第 1 条轴接起来，二维时是左右接	(3, 8)

cat 的要求是：**除了拼接的那条轴，其他轴的长度必须匹配。**

与沿已有轴拼接不同，stack 会新增一条轴：

```
torch.stack((X, Y), dim=0)  # 形状：(2, 3, 4)
```

两种操作的区别是：

• `cat`：沿已有轴接长。

• `stack`：新增一条轴；输入张量形状必须相同。PyTorch stack

两张形状为 `(3, 224, 224)` 的图片，用 `stack(..., dim=0)` 可以组成形状为 `(2, 3, 224, 224)` 的批次。

3.6 点积与矩阵乘法

这类运算经常出现在模型预测中。D2L 2.3 使用 `dot`、`mv`、`mm`：

写法	输入要求与含义	形状示例
`torch.dot(x, y)`	两个等长一维向量，对应相乘后求和	`(4,)` 与 `(4,)` → 标量
`torch.mv(A, x)`	二维矩阵乘一维向量	`(3, 4)` 与 `(4,)` → `(3,)`
`torch.mm(A, B)`	两个二维矩阵相乘	`(3, 4)` 与 `(4, 2)` → `(3, 2)`
`A @ B`	常见的矩阵乘法写法	两个二维输入时，与 `mm` 同类
`torch.matmul(A, B)`	支持向量、矩阵和批次矩阵乘法	高维输入的最后两轴做矩阵乘法，前面的批次轴按广播规则配合

二维矩阵乘法的形状关系为：

```
(m, n) @ (n, p) → (m, p)
    ↑     ↑
    中间两个大小必须相同
```

例如：

```
# 教学变式：10 个样本，每个有 4 个特征
X = torch.randn(10, 4)
W = torch.randn(4, 3)

Y = X @ W               # 输出形状：(10, 3)
```

把每个样本的 4 个输入特征，通过权重组合成 3 个输出。

`X * W` 执行逐元素乘法；在这个示例中，形状 `(10, 4)` 与 `(4, 3)` 不满足广播规则，因此会报错。

除形状外，矩阵乘法还要求输入数据类型和设备兼容；形状正确并不保证计算一定成功。

3.7 比较、筛选与计数

D2L 2.1 出现：

```
X == Y                  # 对应位置分别判断是否相等
```

结果是布尔张量：

```
[[False, True, ...],
 ...]
```

常见补充写法：

写法	作用与含义
`X > 0`	标出哪些位置大于 0
`X == Y`	标出哪些位置相等
`(X == Y).sum()`	统计相等的位置数量
`X[X > 0]`	取出满足“大于 0”的元素

最后一种写法叫布尔索引；这里对整个 `X` 筛选，得到的是一维的选中元素。

准确率计算中的比较与计数通常写成：

```
# 教学变式：pred 和 y 都是类别编号，形状均为 (batch,)
correct = (pred == y).sum()
```

`X == Y` 通常返回多个布尔值。若直接将多元素张量放入 `if` 条件，可能产生“布尔值不明确”的错误；应根据目的使用 `.all()`、`.any()` 或计数。

3.8 综合示例：softmax

下面是 D2L 3.6 的 softmax 实现，添加了中文注释：

```
def softmax(X):
    X_exp = torch.exp(X)                    # 每个分数取指数
    partition = X_exp.sum(1, keepdim=True) # 计算每行的指数总和
    return X_exp / partition               # 每行各自除以自己的总和
```

假设输入形状是 `(32, 10)`，表示 32 个样本，每个有 10 个类别分数：

代码	数据流	形状
`torch.exp(X)`	每个分数分别取指数	`(32, 10)`
`sum(1, keepdim=True)`	把每个样本的 10 个数加起来，保留轴	`(32, 1)`
`X_exp / partition`	利用广播，每行除以自己的总和	`(32, 10)`

把每个样本的类别分数取指数，再按该样本的指数总和归一化，得到每行总和约为 1 的概率。

`keepdim=True` 让分母保持 `(32, 1)`：32 个样本各有一个总和，可以分别除到对应行上。若变成 `(32,)`，形状就无法按预期配合。

这是用于解释原理的基础实现。分数较大时，直接取指数可能溢出。实际计算通常使用框架提供的稳定实现；使用交叉熵时则直接传入原始分数，详见第六章。

下面是教学变式：

```
scores = torch.randn(8, 5)
weights = torch.exp(scores)
totals = weights.sum(dim=1, keepdim=True)
probs = weights / totals
check = probs.sum(dim=1)
```

这段代码的整体作用是：

生成 8 组、每组 5 个分数；取指数，计算各组总和，再分别归一化，最后检查各组概率和。

形状依次是：

```
(8, 5) → (8, 5) → (8, 1) → (8, 5) → (8,)
```

check 中的 8 个数应当都接近 1。

第四章 数据加载与预处理

模型每次通常接收一批样本。本章沿着“原始数据 → 预处理 → 数据集 → 数据加载器”的顺序，解释样本与标签怎样对应，并介绍 D2L 数据代码中的常见 Python 语法。

数据处理的整体流程如下：

```
原始图片、表格或张量
        ↓ 预处理
Dataset：组织单条样本及其标签
        ↓ DataLoader：取样、分批
X：一批输入      y：这一批的正确答案
        ↓
交给模型和损失函数
```

本章以 D2L 3.2、3.3 和 3.5 的数据代码为主，结合 2.2 的表格预处理。代码中的中文注释用于说明操作，教学改写另作标注。

4.1 数据集与数据加载器

写法	作用与含义
`dataset[i]`	取第 `i` 条样本，具体返回什么由数据集定义
`len(dataset)`	数据集有多少条样本，适用于这里这种有长度的数据集
`data.TensorDataset(features, labels)`	把特征张量和标签张量按第 0 条轴对应起来
`data.DataLoader(dataset, batch_size=32)`	把样本组织成每批最多 32 条的数据
`for X, y in data_iter:`	不断取出一批输入和对应标签

```
features.shape = (1000, 2)
labels.shape   = (1000, 1)
```

表示有 1000 个样本，每个样本有两个特征、一个目标值。

```
单条样本：特征 (2,)，标签 (1,)
每批 10 条：X (10, 2)，y (10, 1)
```

`TensorDataset` **不负责把全部数据变成一批，分批主要由** `DataLoader` **完成。** 输入张量的第 0 条轴长度必须一致，否则无法逐条对应。D2L 3.3 读取数据集

4.2 解析 load_array

这是 D2L 3.3 的真实代码，补上相关导入并添加注释：

```
from torch.utils import data

def load_array(data_arrays, batch_size, is_train=True):
    """构造一个PyTorch数据迭代器"""
    dataset = data.TensorDataset(*data_arrays)
    return data.DataLoader(dataset, batch_size, shuffle=is_train)

batch_size = 10
data_iter = load_array((features, labels), batch_size)
```

`load_array` 包含默认参数、元组打包与参数展开三种语法。

`is_train=True`：**默认参数。**

```
load_array((features, labels), 10)
```

没有传第三个参数，就使用 `True`。

```
load_array((features, labels), 10, is_train=False)
```

明确传入 `False`，覆盖默认值。

`(features, labels)`：**把两个对象打包。**

这里把两个张量放进一个元组，一起传给 `data_arrays`。

`*data_arrays`：**调用函数时，把打包的内容展开成多个参数。**

在这个例子里：

```
data.TensorDataset(*data_arrays)
```

相当于：

```
data.TensorDataset(features, labels)
```

函数将特征与标签包装成数据集，再返回数据加载器。`is_train=True` 时打乱样本读取顺序。

D2L 代码中的以下标记属于教材构建流程：

```
#@save
```

这是教材构建工具使用的标记。**对普通 Python 执行来说，它是注释，不表示在运行时保存模型。**

这里的星号发生在函数调用处，表示位置参数展开；它不同于张量乘法中的 `*`。判断含义必须结合语法位置。

4.3 数据加载参数

下面是便于集中认识参数的**教学补充**：

```
loader = data.DataLoader(
    dataset,
    batch_size=32,
    shuffle=True,
    num_workers=0,
    drop_last=False
)
```

参数	作用与含义	容易混淆的地方
dataset	从哪个数据集取样本	不是直接传模型
batch_size=32	每批通常取 32 条	最后一批可能不足 32 条
shuffle=True	每次遍历时打乱样本顺序	不会把一条样本的特征与标签拆散
num_workers=0	在当前进程中加载数据	不是“不加载数据”
num_workers=4	使用 4 个工作进程加载数据	不是使用 4 块 GPU
drop_last=False	保留最后不足一批的数据	默认就是这个行为
drop_last=True	丢弃最后不足一批的数据	会影响每轮使用的样本数

假设有 103 条数据，每批 32 条：

```
drop_last=False: 32、32、32、7, 共 4 批
drop_last=True:  32、32、32, 共 3 批
```

训练数据通常按打乱的顺序读取，测试数据通常顺序读取。`shuffle` 改变样本读取顺序，保持每条样本与其标签的对应关系。PyTorch 数据加载文档

数据集长度与加载器长度分别表示：

```
len(dataset)            # 样本数
len(loader)             # 这里这种加载器的批次数
```

运行环境说明

Windows 使用多个加载工作进程时，通常需要把程序入口放在 `if __name__ == '__main__':` 保护下。初次运行或排查工作进程错误时，可先使用 `num_workers=0`，确认数据读取与预处理本身能正常执行。

4.4 批次解包

这是 Python 的**解包**。

```
for X, y in data_iter:
    ...
```

每次加载器交出一批数据，例如：

```
这一批数据 = [一批特征，一批标签]
```

Python 把第一部分交给 X，第二部分交给 y。

下面是帮助理解的等价改写：

```
for batch in data_iter:
    X = batch[0]
    y = batch[1]
```

for X，y 只有一层循环；每次把同一个批次中的特征与标签分别赋给两个变量。

类似写法还有：

```
train_iter, test_iter = load_data_fashion_mnist(...)
```

函数返回两个对象，分别接收为训练数据加载器和测试数据加载器。

如果返回的部分数量和左边变量数量对不上，就可能出现“解包数量不匹配”的错误。

4.5 迭代器与遍历控制

D2L 3.3 使用：

```
next(iter(data_iter))
```

将调用拆开，可以看出迭代器如何保留遍历状态：

```
# 教学改写
iterator = iter(data_iter)   # 建立一次遍历的状态
X, y = next(iterator)        # 取第一批
X, y = next(iterator)        # 沿着同一个迭代器取下一批
```

写法	含义
iter(data_iter)	开始一次遍历，得到迭代器
next(iterator)	从当前位置继续取下一项
for ... in data_iter	自动重复取数据，直到本次遍历结束
break	立即结束当前这层循环

注意：

```
next(iter(data_iter))
next(iter(data_iter))
```

这两句分别新建了迭代器，不等于顺着同一个迭代器连续取两批。 如果启用了打乱，两次取得的数据还可能不同。

D2L 常这样检查数据：

```
for X, y in train_iter:
    print(X.shape, X.dtype, y.shape, y.dtype)
    break
```

break 不会删除数据，也不会让加载器以后无法继续使用。

4.6 生成器与 yield

D2L 3.2 自己实现过一个分批函数：

```
def data_iter(batch_size, features, labels):
    num_examples = len(features)
    indices = list(range(num_examples))
    random.shuffle(indices)

    for i in range(0, num_examples, batch_size):
        batch_indices = torch.tensor(
            indices[i: min(i + batch_size, num_examples)])
        yield features[batch_indices], labels[batch_indices]
```

这里需要提前导入 `random`、`torch`，并准备好特征与标签。

代码	作用与含义
`len(features)`	取样本总数
`list(range(num_examples))`	生成全部样本编号
`random.shuffle(indices)`	原地打乱这些编号
`range(0, num_examples, batch_size)`	每次前进一个批大小
`indices[i: min(...)]`	取这一批的样本编号，最后一批截到数据末尾
`torch.tensor(...)`	把编号列表变成张量
`features[batch_indices]`	按这些编号取出特征
`labels[batch_indices]`	按同一组编号取出标签
`yield ...`	交出这一批，暂停；下次继续执行

`return` **通常交出结果后结束函数；** `yield` **让函数能够逐次产生结果。**

调用含有 `yield` 的函数会得到生成器，此时函数主体尚未执行。每次迭代推进到下一个 `yield`，交出结果并保留暂停位置，直至函数结束。

相同名称在不同示例中可能指向不同类型的对象：

- 3.2 中的 `data_iter` 是一个函数。
- 3.3 中的 `data_iter` 被用作加载器变量名。

名字相同，不代表对象类型相同，要看它前面怎样定义。 D2L 3.2 分批读取代码

4.7 图像预处理

D2L 3.5 的真实代码：

```
trans = transforms.ToTensor()

mnist_train = torchvision.datasets.FashionMNIST(
    root="../data",
    train=True,
    transform=trans,
    download=True
)
```

相关导入是：

```
import torchvision
from torchvision import transforms
```

逐项读：

写法	作用与含义
`transforms.ToTensor()`	创建一个图片转张量的处理对象
`FashionMNIST(...)`	建立 Fashion-MNIST 数据集
`root="../data"`	数据保存在这个目录中，按当前工作目录解析相对路径
`train=True`	选择训练集
`train=False`	选择测试集
`transform=trans`	取出图片时，对图片应用这个处理
`download=True`	本地缺少所需数据时尝试下载

在这里，`ToTensor` 会把 Fashion-MNIST 的灰度图片转换成浮点张量，像素从 0～255 缩放到 0～1：

```
单张图片 → (1, 28, 28)
一批图片 → (batch_size, 1, 28, 28)
```

这里的 1 是灰度通道数。`train=True` **只是选择训练数据，不会开始训练模型。** D2L 图像分类数据集

D2L 将可选缩放与张量转换组合成预处理流程：

```
trans = [transforms.ToTensor()]

if resize:
    trans.insert(0, transforms.Resize(resize))

trans = transforms.Compose(trans)
```

先准备转张量操作；如果要求调整尺寸，就把缩放操作放在最前面；最后把这些操作串成一个处理流程。

预处理流程的组装方式为：

- [操作]：这里先放进一个 Python 列表。

• `insert(0, 操作)`：把操作插到列表开头。

• `Compose(...)`：按列表顺序依次执行。

• `resize=None`：常表示没有指定调整尺寸；这里用 `if resize:` 判断是否执行。

顺序有含义：这段代码安排的是**先调整尺寸，再转成张量。**

4.8 表格预处理

D2L 2.2 中的典型写法：

教材写法	作用与含义
`pd.read_csv(data_file)`	读取 CSV 表格
`data.iloc[:, 0:2]`	按位置取全部行、前两列
`data.iloc[:, 2]`	按位置取全部行、第 3 列
`inputs.fillna(...)`	用指定值填补缺失项
`pd.get_dummies(inputs, dummy_na=True)`	把类别列展开成指示列，并为缺失类别提供指示列
`inputs.to_numpy(dtype=float)`	把表格转换成浮点数数组
`torch.tensor(...)`	再把数组转换成 PyTorch 张量

教材的真实片段：

```
inputs, outputs = data.iloc[:, 0:2], data.iloc[:, 2]
inputs = inputs.fillna(inputs.mean())
inputs = pd.get_dummies(inputs, dummy_na=True)

X = torch.tensor(inputs.to_numpy(dtype=float))
y = torch.tensor(outputs.to_numpy(dtype=float))
```

划分输入列和目标列，给数值缺失项填均值，把类别转成指示列，再转换为张量。

教材中的 `inputs.mean()` 在包含文字列的较新 pandas 环境中可能报错；兼容性改写可以是：

```
# 教学补充：只对数值列求均值
inputs = inputs.fillna(inputs.mean(numeric_only=True))
```

另外，`to_numpy(dtype=float)` 产生的浮点类型不一定是模型常用的 `float32`。**“已经是张量”不等于“形状和类型已经满足模型要求”。** D2L 数据预处理、pandas 均值参数

4.9 常见 Python 语法

列表推导式：逐个处理，组成新列表。

D2L 的标签转换函数中有：

```
return [text_labels[int(i)] for i in labels]
```

遍历每个数字标签，转成整数索引，查出类别名称，组成一个列表返回。

阅读列表推导式时，可先找迭代部分 `for i in labels`，再看每个元素如何映射为输出。

`zip` 加 `enumerate`：配对，再编号。

D2L 的图片显示代码中有：

```
for i, (ax, img) in enumerate(zip(axes, imgs)):
```

把画图区和图片一一配对，同时取得当前配对的编号。

- `zip(axes, imgs)`：每次交出一个（画图区，图片）。
- `enumerate(...)`：再加上默认从 0 开始的编号。
- `i, (ax, img)`：按相同结构接收。

这是 Python 语法，并非 PyTorch 专属功能。D2L 标签与图片显示代码

`with`：在缩进范围内使用一种上下文。

D2L 3.2 的片段：

```
with torch.no_grad():
    train_l = loss(net(features, w, b), labels)
    print(f'epoch {epoch + 1}, loss {float(train_l.mean()):f}')
```

这段代码的作用是：

在不记录求梯度信息的范围内，计算并打印训练损失。

`with` 的效果由后面的对象决定。这里是 `torch.no_grad()`；不能看到所有 `with` 都理解成关闭梯度。

`f'...'` 是字符串格式化：把 `{...}` 中表达式的结果填进字符串；`:f` 表示按定点小数格式显示。

4.10 综合示例：检查图像批次

D2L 3.5 的真实调用片段：

```
train_iter, test_iter = load_data_fashion_mnist(32, resize=64)

for X, y in train_iter:
    print(X.shape, X.dtype, y.shape, y.dtype)
    break
```

建立训练和测试数据加载器，每批 32 张图片，把图片调整到 64×64；取一批训练数据，查看输入和标签的形状及类型，然后结束检查循环。

这一批的形状是：

```
X: (32, 1, 64, 64) —— 32 张单通道图片
y: (32,)           —— 每张图片对应一个类别编号
```

这段代码完成数据准备、分批和检查，没有执行模型训练。`resize=64` 改变了输入空间尺寸；后续模型的特征数也必须与之对应。

第五章 模型定义与参数

读模型代码，主要看三件事：有哪些层、数据怎样经过这些层、哪些数需要训练。本章先介绍顺序模型和自定义模型，再通过 LeNet 观察每层怎样改变数据形状。

```
创建层：nn.Linear(...)、nn.Conv2d(...)
连接计算：nn.Sequential(...) 或 forward(...)
获取参数：net.parameters()
```

本章结合 D2L 4.3、5.1、5.2 与 6.6 的 PyTorch 实现。需要特别区分创建层对象、调用层进行计算，以及更新层中的参数。

5.1 创建模型与调用模型

写法	作用与含义
`nn.Linear(20, 256)`	创建一个输入 20 个特征、输出 256 个特征的全连接层
`nn.ReLU()`	创建一个 ReLU 激活层
`MLP()`	根据 `MLP` 这个类，创建一个模型对象
`net(X)`	把数据 `X` 送入已经创建的模型
`layer(X)`	把 `X` 送入某一层
`print(net)`	查看模型结构

例如：

```
layer = nn.Linear(20, 256)   # 创建层，此时还没有处理输入数据
Y = layer(X)                 # 使用这层计算结果
```

定义网络结构和参数，不等于训练；调用模型做预测，也不等于更新参数。

5.2 顺序容器 Sequential

这是 D2L 4.3 多层感知机的真实模型：

```
net = nn.Sequential(
    nn.Flatten(),          # 将每个样本展开
    nn.Linear(784, 256),   # 784 个输入特征 → 256 个输出特征
    nn.ReLU(),             # 激活
    nn.Linear(256, 10)     # 256 个特征 → 10 个类别分数
)
```

假设输入是一批 Fashion-MNIST 图片：

经过哪一步	形状
输入	`(32, 1, 28, 28)`
`Flatten`	`(32, 784)`
第一个 `Linear`	`(32, 256)`
`ReLU`	`(32, 256)`
第二个 `Linear`	`(32, 10)`

每张图片先展开成 784 个数，再转换成 256 个特征，经过激活，最后得到 10 个类别分数。

形状与数值含义分别为：

- `32` 是批大小，不是写在 `Linear` 中的特征数。
- `784 = 1 × 28 × 28`。
- `ReLU` 改数值，不改变这里的形状。
- 输出的 10 个数是类别分数，**还不能直接称为概率**。

ReLU 把负数变为 0，非负数保持不变。它让多层网络能够表达更复杂的关系；如果层与层之间只有线性计算，叠加多层也仍然只是线性变换。

如果图片改成 64×64，而第一层仍要求 784 个输入特征，可能报矩阵形状不匹配。

5.3 全连接层的参数与计算

补充一个具体例子：

```
layer = nn.Linear(4, 3)
```

这层默认有：

```
weight.shape = (3, 4)
bias.shape   = (3,)
```

虽然构造时写的是“输入 4、输出 3”，权重存储形状却是：

```
(输出特征数，输入特征数)
```

对于二维批次输入，可以这样理解其计算：

```
# 教学表达：解释计算关系
Y = X @ layer.weight.T + layer.bias
```

形状对应：

```
X:         (10, 4)
weight.T:  (4, 3)
bias:      (3,)

Y:         (10, 3)
```

参数数量是：

```
权重：3 × 4 = 12
偏置：3
总计：15 个数
```

`Linear` **负责做加权计算；它本身不会自动包含 ReLU 等激活函数。**

5.4 自定义模型 Module

D2L 5.1 层和块的真实实现：

```
from torch import nn
from torch.nn import functional as F

class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(20, 256)
        self.out = nn.Linear(256, 10)

    def forward(self, X):
        return self.out(F.relu(self.hidden(X)))
```

各个语法部分的职责如下：

写法	作用与含义
`class MLP(nn.Module):`	定义一种模型，继承 PyTorch 的模型基础功能
`__init__`	创建模型对象时，准备它需要的层和状态
`self`	当前这个模型对象
`super().__init__()`	初始化父类，让层和参数管理等机制正常工作
`self.hidden = ...`	创建隐藏层，并把它保存为模型的一部分
`self.out = ...`	创建输出层
`forward(self, X)`	规定输入数据怎样经过这些层
`return ...`	返回模型输出

嵌套调用可按照由内到外的顺序展开：

```
return self.out(F.relu(self.hidden(X)))
```

相当于下面这个**教学改写**：

```
H = self.hidden(X)      # (batch, 20) → (batch, 256)
H = F.relu(H)           # 激活，形状不变
Y = self.out(H)         # (batch, 256) → (batch, 10)
return Y
```

自定义模型时需要注意：

• 遗漏 `super().__init__()`，可能在给模型添加子层时就报错。

• 把需要持续学习的层每次都在 `forward` 里重新创建，会不断得到新参数，无法按预期积累训练结果。

`__init__` 通常负责“准备有哪些层”，`forward` 负责“这次怎么使用这些层”。

5.5 模型调用与函数式接口

D2L 的调用方式是：

```
net = MLP()             # 创建模型
net(X)                  # 计算输出
```

`net(X)` 会调用模型定义的 `forward`，并执行 PyTorch 配套的处理。直接调用 `net.forward(X)` 会绕过其中一部分处理，因此正常使用模型时写 `net(X)`。

模型的标准使用方式是：

自己定义 `forward`，使用时调用 `net(X)`。

类、层对象与函数调用的区别如下：

写法	得到什么
`nn.ReLU`	ReLU 这个类
`nn.ReLU()`	一个可以放进模型的层对象
`F.relu(X)`	立刻对张量 X 做 ReLU，返回结果

`nn.ReLU()(X)` 与 `F.relu(X)` 都对输入执行 ReLU。前者先创建层对象再调用，后者直接调用函数；顺序容器通常使用层对象，自定义 `forward` 中两种形式都常见。

5.6 卷积、激活与池化

D2L 6.6 LeNet中有这些写法：

写法	作用与含义
`nn.Conv2d(1, 6, kernel_size=5, padding=2)`	输入 1 通道，输出 6 通道，用 5×5 卷积核，边缘填充 2
`nn.Conv2d(6, 16, kernel_size=5)`	输入 6 通道，输出 16 通道，用 5×5 卷积核
`nn.Sigmoid()`	对每个数做 Sigmoid 激活，形状不变
`nn.AvgPool2d(kernel_size=2, stride=2)`	用 2×2 窗口做平均池化，每次移动 2 格
`nn.Flatten()`	默认保留批次轴，把后面的轴展平

卷积层的主要参数为：

参数	意思
`in_channels`	输入通道数
`out_channels`	输出通道数
`kernel_size`	卷积核的空间大小
`stride`	每次移动多少格
`padding`	在输入边缘补多少
`bias`	是否包含偏置

图像通道可以理解为同一张图的不同数值层：灰度图通常有 1 个通道，RGB 彩图有 3 个通道；卷积得到的多个通道则表示不同的特征图。

`Conv2d` 中省略的常用默认值包括 `stride=1`、`padding=0`。

输出通道数不是图片数量。 输入 32 张图片，变成 6 通道后，仍然是 32 个样本，只是每个样本有 6 张特征图。

池化层通常没有可学习参数。作为补充，`nn.MaxPool2d(...)` 的窗口中取最大值，平均池化则取平均值。

卷积与池化的空间尺寸

本例卷积步长为 1，使用普通卷积核。对高或宽，有“输出大小 = 输入大小 + 两侧填充总量 - 卷积核大小 + 1”。所以第一次是 `28 + 2 * 2 - 5 + 1 = 28`，第二次是 `14 - 5 + 1 = 10`。这里的 2×2、步长 2 池化把尺寸减半：28 变为 14，10 变为 5。

5.7 综合示例：LeNet

这是教材真实定义，添加中文注释：

```
net = nn.Sequential(
    nn.Conv2d(1, 6, kernel_size=5, padding=2), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2),

    nn.Conv2d(6, 16, kernel_size=5), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2),

    nn.Flatten(),
    nn.Linear(16 * 5 * 5, 120), nn.Sigmoid(),
    nn.Linear(120, 84), nn.Sigmoid(),
    nn.Linear(84, 10)
)
```

输入为 28×28 单通道图片时，各层形状如下；B 表示批大小：

操作	输出形状
输入	(B, 1, 28, 28)
第一次卷积、激活	(B, 6, 28, 28)
第一次池化	(B, 6, 14, 14)
第二次卷积、激活	(B, 16, 10, 10)
第二次池化	(B, 16, 5, 5)
展平	(B, 400)
全连接、激活	(B, 120)
全连接、激活	(B, 84)
输出层	(B, 10)

两组卷积提取特征、池化缩小空间尺寸，再把特征展平，通过全连接层输出 10 个类别分数。

全连接层的输入特征数由前一层输出决定：

```
nn.Linear(16 * 5 * 5, 120)
```

16 * 5 * 5 对应的含义是：

上一步每个样本有 16 个通道，每个通道是 5×5，展开后就是 400 个特征。

如果前面的输入尺寸、卷积或池化改变了，这个 400 也可能需要跟着改。

5.8 参数访问与管理

D2L 5.2 参数管理经常使用下面这些接口：

写法	作用与含义
net.parameters()	遍历模型登记的参数
net.named_parameters()	同时取得参数名字和参数对象
layer.weight	访问这一层的权重
layer.bias	访问这一层的偏置
param.shape	查看这组参数的形状
param.grad	查看已经计算出的梯度
net.state_dict()	获取模型状态字典，包含参数及需要保存的其他状态

例如，下面是便于阅读的**教学改写**：

```
for name, param in net.named_parameters():
    print(name, param.shape)
```

parameters() 逐个产生参数对象。每个对象通常是一整块权重矩阵或偏置向量，内部可包含许多标量。

对于 Sequential，还可以按编号访问层。以前面的 4.3 模型为例：

```
net[0]                  # Flatten
net[1]                  # 第一个 Linear
net[1].weight           # 这个 Linear 的权重
```

编号包含没有参数的层。 不要因为 Flatten 没有权重，就以为它不占编号。

param.data 访问参数的数据部分，会绕开自动求导的一些保护。手动初始化或修改参数时，应优先使用初始化接口，或在 torch.no_grad() 范围内进行必要操作。

5.9 参数初始化

D2L 4.3 的真实片段：

```
def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight, std=0.01)

net.apply(init_weights)
```

初始化函数的执行逻辑如下：

代码	意思
def init_weights(m)	定义一个处理模型模块的函数
type(m) == nn.Linear	判断这个模块是否恰好是全连接层
m.weight	取出它的权重
nn.init.normal_(..., std=0.01)	用默认均值 0、标准差 0.01 的正态随机数填入权重
net.apply(init_weights)	对模型及其子模块逐个调用这个函数，包括嵌套的子模块

以下细节决定了这段初始化代码的行为：

1. normal_ **末尾的下划线表示原地修改传入的张量。**

2. apply(init_weights) 传入的是函数；不是先写 init_weights() 调用一次。

3. 这段只重新初始化权重，没有重新设置偏置。

D2L 还会出现这些初始化写法：

写法	用途
nn.init.zeros_(m.bias)	把偏置设为 0
nn.init.constant_(m.weight, 1)	把权重设成指定常数
nn.init.xavier_uniform_(m.weight)	按 Xavier 规则随机初始化权重

初始化决定训练的起点。Linear、Conv2d 等层在创建时已执行默认初始化；再次调用初始化函数，是用指定规则覆盖原有初值。

5.10 逐层检查输出形状

D2L 6.6 使用这段真实代码：

```
X = torch.rand(size=(1, 1, 28, 28), dtype=torch.float32)

for layer in net:
    X = layer(X)
    print(layer.__class__.__name__, 'output shape: \t', X.shape)
```

生成一张随机的单通道图片，按顺序通过每一层，打印层类型和输出形状。

写法	意思
`for layer in net`	依次取出 Sequential 中的层
`X = layer(X)`	用上一层结果作为这一层输入，再接收新结果
`layer.__class__.__name__`	取层的类型名称，例如 Conv2d
`X.shape`	查看这一步输出的形状

这里没有反向传播，也没有参数更新，它是在**检查前向数据流。**

第六章 自动求导、损失与优化器

训练的核心是：用损失衡量预测误差，用梯度判断参数怎样调整，再由优化器执行更新。本章重点分清这三步，以及为什么每批通常要先清除旧梯度。

写法	作用与含义
`pred = net(X)`	根据当前参数做预测
`l = loss(pred, y)`	比较预测和正确答案，计算损失
`trainer.zero_grad()`	清除上一次留下的梯度
`l.backward()`	计算损失对参数的梯度
`trainer.step()`	根据梯度调整参数

这五句分别对应前向计算、损失计算、清理梯度、反向传播和参数更新。理解它们的先后关系，比记住变量名称更重要。

6.1 开启梯度追踪

用途：让 PyTorch 知道后面需要对哪些张量求梯度。

D2L 2.5 自动微分中的真实语句，合并相邻单元并添加注释：

```
x = torch.arange(4.0)    # [0., 1., 2., 3.]
x.requires_grad_(True)   # 开启关于 x 的梯度追踪

y = 2 * torch.dot(x, x)  # 计算一个标量
y.backward()             # 计算 y 对 x 的梯度
x.grad                   # 查看结果
```

写法	含义
`x.requires_grad`	查看是否需要梯度追踪
`x.requires_grad_(True)`	原地开启
`torch.tensor(..., requires_grad=True)`	创建时就开启
`x.grad`	查看已经存下来的梯度
`y.grad_fn`	查看产生 y 的求导节点信息

这个例子中：

```
x.shape:       (4,)
y.shape:       (), 标量
x.grad.shape:  (4,)
```

梯度结果是：

```
[0., 4., 8., 12.]
```

这里 y 等于各分量平方和的两倍，因此第 i 个分量的梯度为 4 * x[i]。梯度与 x 形状相同，描述各分量对结果的局部影响。

梯度追踪有以下前提：

• 开启追踪后，x.grad 不会立刻出现数值；还没有反向计算时，它通常是 None。

• 整数张量不能像浮点张量这样开启梯度。

• 训练时通常不需要给输入图片开启梯度；模型权重需要梯度，就可以计算权重的更新方向。

nn.Linear 等层的参数通常默认已经开启梯度追踪，所以模型代码里不一定能看到手动设置。

6.2 反向传播与参数梯度

```
l.backward()
```

沿着产生损失 l 的计算关系，反向计算相关参数的梯度。

通常计算链条是：

```
参数 → 模型输出 → 损失
                  ↓ backward
参数的 .grad ← 反向计算
```

参数值与参数梯度分别通过以下对象访问：

```
param                   # 参数本身
param.grad              # 损失对这个参数的梯度
```

如果权重形状是 (256, 784)，它的梯度通常也是 (256, 784)，每个权重对应一个梯度值。

读取 .grad 时，需要注意哪些结果会保存梯度：

模型参数这类直接创建、需要求导的张量，通常会保存 .grad；由其他张量计算出的中间结果，默认不一定保存。前一类常称为“叶子张量”。

所以某个中间结果的 .grad 是 None，不一定表示梯度计算失败。

6.3 单元素目标与损失汇总

用途：先把多个损失汇总成一个目标，再求梯度。

D2L 2.5 的真实写法：

```
x.grad.zero_()           # 此处前面的计算已产生 x.grad
y = x * x               # 每个元素各自平方
y.sum().backward()       # 先加成一个数，再求梯度
```

这里：

```
y:        (4,)
y.sum():  (), 标量
```

常见写法可以这样区分：

写法	作用与含义
`l.backward()`	对已经汇总好的单元素损失求梯度
`l.sum().backward()`	对所有损失的总和求梯度
`l.mean().backward()`	对所有损失的平均值求梯度

如果 `l` 包含多个元素，直接调用不带额外参数的 `backward()`，通常会报“只能对标量输出隐式创建梯度”之类的错误。

`backward()` **不会自动对一批损失求平均。** 损失的汇总方式应由调用代码明确决定。

另外，同一条计算图反向传播后，相关中间信息通常会释放。如果对同一个损失再次调用 `backward()`，可能报“重复反向传播”的错误。普通训练每一批会重新前向计算，得到新的计算图。

6.4 梯度清理

D2L 2.5 明确使用：

```
x.grad.zero_()
```

它把已有梯度张量中的数值设为 0。

模型训练中更常见：

```
trainer.zero_grad()
```

写法	作用范围
`x.grad.zero_()`	清零一个已经存在的梯度张量
`trainer.zero_grad()`	清除该优化器管理的参数梯度
`net.zero_grad()`	清除模型登记的参数梯度

为什么需要它?

```
第一次 backward：存下第一批的梯度
第二次 backward：默认继续累加
```

如果希望每批独立更新，就要在这批反向传播前清除旧梯度。

清理梯度时需要注意：

- **清的是梯度，不是模型参数，也不是输入数据。**
- `zero_grad()` 可将梯度设为 `None`，也可把已有梯度张量清零，具体行为取决于版本和 `set_to_none` 参数。两者都用于清理旧梯度，但 `None` 与零张量并非同一种状态。PyTorch 梯度清理接口

如果 `x.grad` 本来就是 `None`，直接写 `x.grad.zero_()` 会报错。这也是训练时使用优化器接口更方便的原因之一。

6.5 分离结果与关闭梯度记录

写法	作用与含义
`u = y.detach()`	得到一个与 y 数值相同、但脱离原计算图的张量
`with torch.no_grad():`	这个范围内的计算不记录反向求导关系
`l.item()`	把单元素张量取成普通 Python 数字

D2L 2.5 的真实片段：

```
x.grad.zero_()
y = x * x
u = y.detach()
z = u * x

z.sum().backward()
```

y 由 x 算出来，但通过 u 参与后续计算时，把它当成常数。

计算路径可以这样读：

```
x → y → detach → u
                 ↓ 当常数使用
x ————————————→ u * x → 损失
```

`detach` **只切断经过它的梯度路径，并不意味着后面所有计算都不能求导。** 这里 z 仍然直接依赖 x。

这三个操作不能相互替代：

• `detach()` 不等于复制数据；返回张量与原张量共享存储。

• `l.item()` 返回普通数字，不能再写 `l.item().backward()`。

记录损失时，可以把单元素张量转换为普通数字：

```
# 教学补充
loss_value = l.item()    # 用来打印、记录
```

但训练反向传播仍然使用原来的张量 `l`。

6.6 损失函数与损失值

D2L 3.3 使用：

```
loss = nn.MSELoss()      # 创建计算损失的工具
l = loss(net(X), y)      # 使用工具，得到这批的损失值
```

可以拆成下面的**教学改写**：

```
pred = net(X)
l = loss(pred, y)
```

名字	在这里代表什么
`loss`	损失函数对象

名字	在这里代表什么
pred	模型预测张量
y	正确答案张量
l	计算得到的损失张量

其他项目可能把它们命名为 criterion、loss_fn、loss_value。**判断含义要看赋值来源，不能只认变量名。**

回归与多类别分类中常见的损失接口如下：

写法	常见用途
nn.MSELoss()	平方误差，常用于连续数值预测
nn.CrossEntropyLoss()	交叉熵，常用于多类别分类

D2L 3.2 还手写过平方损失：

```
def squared_loss(y_hat, y):
    return (y_hat - y.reshape(y_hat.shape)) ** 2 / 2
```

把标签形状整理得与预测一致，逐个计算预测误差，平方，再除以 2。

手写平方损失具有以下特征：

• y_hat 是预测值。

• reshape 防止本来应一一对应的数发生意外广播。

• 这个函数返回逐项损失，没有自己取平均。

• 它含有 / 2，所以数值上不等于默认的 nn.MSELoss()。D2L 从零实现、简洁实现

6.7 损失汇总参数 reduction

D2L 4.3 使用：

```
loss = nn.CrossEntropyLoss(reduction='none')
```

reduction 表示如何汇总损失。

参数	意思
reduction='none'	不自动汇总
reduction='sum'	求和
reduction='mean'	按该损失函数的规则取平均，通常是默认值

对于普通分类，假设输入是 (32, 10)，每个样本对应一个类别编号：

```
reduction='none': 结果形状 (32,)
reduction='sum':  结果为标量
reduction='mean': 结果为标量
```

结合损失的汇总方式，可以解释下面的写法：

```
l = loss(net(X), y)
l.mean().backward()
```

其含义为：

先获得每个样本的损失，再对平均损失求梯度。

不能看到 `loss(...)` 就认定它只返回一个数，要看 `reduction` 和损失类型。

普通无权重情形下，求和与求平均的梯度相差元素数量这个倍数；这会影响更新幅度，不能随意互换。

6.8 交叉熵的输入要求

D2L 4.3 的模型末尾是：

```
nn.Linear(256, 10)
```

训练使用：

```
loss = nn.CrossEntropyLoss(reduction='none')
```

在这种“类别编号作为标签”的常见写法中：

```
预测：浮点张量 (batch, 类别数)
标签：整数张量 (batch,)
```

例如：

```
预测形状：(32, 10)
标签形状：(32,)
标签内容：0 到 9 的类别编号
```

使用类别编号标签时，应遵循以下规则：

- `CrossEntropyLoss` **接收原始类别分数，通常不用提前做 softmax。**
- **不要先** `argmax` **得到类别编号，再拿它算这种交叉熵。**

`argmax` 适合之后“判断预测类别”；算损失时需要保留各类别的分数。

本例标签应为 `torch.long` 类型，类别编号范围为 `0` 到 `类别数 - 1`。类型错误、编号越界或形状不匹配都可能导致报错。该接口也支持概率标签，但其要求不同，不适用于这里的类别编号说明。PyTorch 交叉熵接口

6.9 优化器的职责

D2L 的真实写法：

```
trainer = torch.optim.SGD(net.parameters(), lr=lr)
```

部分	含义
`torch.optim`	优化器模块
`SGD`	使用随机梯度下降类算法
`net.parameters()`	指定要管理哪些参数

部分	含义
lr=lr	左边是参数名，右边是当前变量的值
trainer.step()	根据已有梯度执行一次更新

补充认识一种常见写法：

```
# 教学补充
optimizer = torch.optim.Adam(net.parameters(), lr=0.001)
```

Adam 与 SGD 都接收待更新参数和学习率，但使用不同的更新规则。二者在训练循环中通常都通过 zero_grad() 清理梯度、通过 step() 执行更新。

注意：

• 创建优化器不等于开始更新。

• 本章的 SGD 和 Adam 调用 step() 时使用已有梯度，不会自动执行 backward()。

• 优化器通常不会自动清除已用过的梯度。

• net.parameters() 中有参数，不代表这次一定更新它；还要有可用梯度等条件。

6.10 手写 SGD 与梯度缩放

D2L 3.2 的真实实现：

```
def sgd(params, lr, batch_size):
    """小批量随机梯度下降"""
    with torch.no_grad():
        for param in params:
            param -= lr * param.grad / batch_size
            param.grad.zero_()
```

语句	作用与含义
with torch.no_grad()	更新参数这段操作不再建立求导关系
for param in params	依次处理每组参数
param.grad	取出已经算好的梯度
lr * param.grad	用学习率控制调整幅度
param -= ...	沿负梯度方向修改参数
param.grad.zero_()	清掉这次使用过的梯度

这里除以 batch_size，是为了与前面的损失汇总方式配合：

因为教材在前面用了：

```
l.sum().backward()
```

算的是一批损失**总和**的梯度，所以更新时再除以批大小。

如果已经对平均损失求梯度，普通 SGD 更新时通常就不应再机械地除一次批大小。 读代码要把“损失怎样汇总”和“参数怎样更新”连起来看。

no_grad 关闭梯度记录，允许在不构建反向计算图的情况下修改参数。手动更新参数正是这一用途。

6.11 综合示例：单批参数更新

下面是 D2L 3.3 训练循环中的真实内层片段：

```
for X, y in data_iter:
    l = loss(net(X), y)
    trainer.zero_grad()
    l.backward()
    trainer.step()
```

假设这里是线性回归，使用默认 nn.MSELoss()：

```
X:                (10, 2)
net(X):           (10, 1)
y:                (10, 1)
l:                标量
每组参数的 .grad:  与该参数形状一致
```

取出一批数据，预测并计算平均平方误差，清除旧梯度，计算这次的梯度，再按梯度调整参数。

执行 step() 后，模型参数得到更新；下一批预测将使用这些新值。输入样本与标签不会因此被修改。

清零也可能放在前向计算之前。对这种普通单批更新流程，关键要求是：**在本批 backward() 前清除上次梯度，在计算出本批梯度之后再 step()。**

第七章 训练与评估

完整训练程序会反复处理多批数据，并统计每一轮的结果。本章展开 D2L 的训练函数，分清哪些语句在更新参数，哪些在评估、计数和显示结果。

训练与评估的整体流程如下：

```
每一轮 epoch：
    切换到训练模式
    逐批：预测 → 算损失 → 清梯度 → 求梯度 → 更新参数
    汇总本轮训练指标

    切换到评估模式
    逐批预测，统计评估指标
    显示或绘制结果

开始下一轮
```

本章主要展开 D2L 3.6 的训练和评估代码，并对照 6.6 的统计方式。

7.1 训练模式与评估模式

写法	作用与含义
`net.train()`	把模型及其子模块切到训练模式
`net.eval()`	切到评估模式
`net.training`	查看当前是否处于训练模式
`with torch.no_grad():`	这段计算不记录反向求导关系

`train()` 的调用形式如下：

```
net.train()
```

它不会自动读取数据、计算损失或更新参数。 它只是切换模式。

模式会影响某些层，例如：

• Dropout：训练时随机丢弃部分激活，评估时停止这种随机丢弃。

• BatchNorm：通常在训练和评估时采用不同的统计方式。

D2L 的 Dropout 模型里就有这样的真实片段：

```
if self.training == True:
    H1 = dropout_layer(H1, dropout1)
```

只有处于训练模式，才执行这里的 Dropout。D2L 4.6 暂退法

模型模式与梯度记录分别由两个开关控制：

```
net.eval()

with torch.no_grad():
    y_hat = net(X)
```

• `eval()`：改变相关层的行为。

• `no_grad()`：控制求导记录。

`eval()` **不会自动关闭梯度；**`no_grad()` **也不会自动切换评估模式。**

7.2 轮次、批次与更新步

术语	常见意思
batch	一批数据
epoch	完整遍历一次训练数据
step	一次参数更新，或代码中自行定义的计数单位

例如有 1000 条数据，每批 100 条：

```
一轮：10 批
每批更新一次：一轮 10 次更新
训练 3 轮：总共 30 次更新
```

对应代码结构：

```
# 教学示意
for epoch in range(num_epochs):     # 控制轮次
    for X, y in train_iter:         # 控制本轮的各批数据
        ...                         # 处理这一批
```

注意缩进：

```
for epoch in range(num_epochs):
    for X, y in train_iter:
        ...                         # 每批执行

    ...                             # 每轮执行
```

在普通 `DataLoader` 中，新一轮的 `for` 会重新开始遍历。模型和优化器通常在外面创建，**不会因为进入新一轮就重新初始化参数。**

7.3 预测类别与 argmax

用途：分类模型输出每个类别的分数，需要选出预测类别。

下面是教学例子：

```
y_hat = torch.tensor([
    [0.1, 0.7, 0.2],
    [0.8, 0.1, 0.1]
])

pred = y_hat.argmax(dim=1)
```

结果：

```
pred = [1, 0]
```

逐行解释：

• 第一行最大的数在索引 1，所以预测为第 1 类。

• 第二行最大的数在索引 0，所以预测为第 0 类。

形状变化：

```
类别分数：(batch，类别数)
            ↓ argmax(dim=1)
类别编号：(batch,)
```

argmax **返回最大值所在的索引，不是最大值本身。**

D2L 经常写：

```
y_hat.argmax(axis=1)
```

在这里可以按常见写法 argmax(dim=1) 理解。

分类时，为了选出最大分数的类别，通常不必先做 softmax；softmax 不会改变正常有限分数之间的大小顺序。

7.4 正确数量与准确率

这是 D2L 3.6 的真实实现：

```
def accuracy(y_hat, y):
    """计算预测正确的数量"""
    if len(y_hat.shape) > 1 and y_hat.shape[1] > 1:
        y_hat = y_hat.argmax(axis=1)
    cmp = y_hat.type(y.dtype) == y
    return float(cmp.type(y.dtype).sum())
```

按顺序读：

代码	意思
len(y_hat.shape) > 1	预测结果不止一条轴
y_hat.shape[1] > 1	每个样本有多个类别分数
argmax(axis=1)	把分数转换成类别编号
type(y.dtype)	转成与标签相同的数据类型
== y	对应比较预测类别与真实类别
.sum()	数出预测正确的数量
float(...)	转成普通 Python 数字

例如：

```
预测：[1，0，2，1]
标签：[1，2，2，0]
比较：[真，假，真，假]
```

函数返回 2.0。

准确率还要除以样本数：

```
accuracy(y_hat, y) / len(y)  # 得到 0.5
```

不要只凭函数名字猜返回值，要看最后的 `return`。

这里针对教材的普通多分类任务，标签通常应是 `(batch,)`。如果标签误写成 `(batch, 1)`，比较时可能发生错误的广播。这段函数也不是通用的二分类阈值判断函数。

7.5 指标累加器

D2L 用它累计整轮结果：

```
metric = Accumulator(3)
```

这表示建立三个累计位置。具体放什么，由调用代码约定。

训练中通常是：

```
metric[0]：损失总和
metric[1]：预测正确的数量
metric[2]：样本总数
```

写法	作用与含义
`Accumulator(3)`	准备三个累计位置，初始为 0
`metric.add(a, b, c)`	把三个值分别加到对应位置
`metric[0]`	读取第一个累计值
`metric.reset()`	把累计值重置为 0

据此可写为：

```
metric[0] / metric[2]    # 平均每个样本的损失
metric[1] / metric[2]    # 整体准确率
```

指标累计不是梯度累计。 `metric.add(...)` 不会替模型求梯度或更新参数。

再留意创建位置：

```
for epoch in range(num_epochs):
    metric = Accumulator(3)
```

表示每轮重新统计。若放到轮次循环外面，统计范围就会变成多轮累计。

下面是 D2L 3.6 的累加器实现。`add` 把新值加到对应位置，`reset` 清空计数，`__getitem__` 让对象能用 `metric[i]` 读取。

```
class Accumulator:
    """在 n 个变量上累加。"""
    def __init__(self, n):
        self.data = [0.0] * n  # 创建 n 个累计位置

    def add(self, *args):
        self.data = [a + float(b) for a, b in zip(self.data, args)]
```

```
    def reset(self):
        self.data = [0.0] * len(self.data)  # 清空所有累计值

    def __getitem__(self, idx):
        return self.data[idx]  # 支持 metric[idx]
```

7.6 解析单轮训练函数

这是 D2L 3.6 的 PyTorch 实现，保留逻辑并添加中文注释：

```
def train_epoch_ch3(net, train_iter, loss, updater):
    # 如果是 PyTorch 模型，切换到训练模式
    if isinstance(net, torch.nn.Module):
        net.train()

    # 累计：损失总和、正确数量、样本数
    metric = Accumulator(3)

    for X, y in train_iter:
        y_hat = net(X)              # 预测
        l = loss(y_hat, y)          # 计算逐样本损失

        if isinstance(updater, torch.optim.Optimizer):
            updater.zero_grad()    # 内置优化器：清梯度
            l.mean().backward()     # 对平均损失求梯度
            updater.step()         # 更新参数
        else:
            l.sum().backward()      # 自定义更新器：先对损失和求梯度
            updater(X.shape[0])     # 传入本批大小，完成更新

        metric.add(float(l.sum()), accuracy(y_hat, y), y.numel())

    return metric[0] / metric[2], metric[1] / metric[2]
```

切换训练模式，逐批预测并计算损失，根据更新器类型完成参数更新，同时累计损失、正确数量和样本数，最后返回本轮平均损失与准确率。

几个新写法集中认识：

写法	作用
`isinstance(net, torch.nn.Module)`	判断 net 是不是 PyTorch 模型，也接受其子类对象
`isinstance(updater, torch.optim.Optimizer)`	判断更新器是不是内置优化器
`updater(X.shape[0])`	调用自定义更新函数，传入当前批的实际样本数
`return a, b`	返回两个值，调用者可以分别接收

D2L 的从零实现允许 net 是普通 Python 函数，此时它没有 `.train()` 方法。因此，函数先检查对象类型，再决定是否切换模式。

这段函数要求损失能按样本统计。 教材使用 `reduction='none'`，因此 `l.sum()` 才是整批损失总和。若改为默认平均损失，统计部分也必须相应修改，否则返回的平均损失不正确。

7.7 整轮指标的加权统计

D2L 6.6 的训练代码使用默认交叉熵：

```
loss = nn.CrossEntropyLoss()
```

此时 l 已经是这一批的平均损失，所以教材这样累计：

```
with torch.no_grad():
    metric.add(l * X.shape[0], d2l.accuracy(y_hat, y), X.shape[0])
```

平均损失乘以本批样本数，得到本批损失总和，再累计正确数量与样本数。

对这里普通、无加权的分类任务：

l 是什么	怎样获得本批损失总和
每个样本一个损失	l.sum()
一批样本的平均损失	l * 本批样本数

最后再统一除以总样本数。

各批样本数可能不同，因此整轮准确率不能简单地对每批准确率取算术平均。

例如：

```
第一批：100 个样本，正确率 90%
第二批：  10 个样本，正确率  0%
```

整体准确率是：

```
90 / 110 ≈ 81.8%
```

不是：

```
(90% + 0%) / 2 = 45%
```

整轮统计通常累计“总正确数、总样本数”，最后再相除。 D2L LeNet 训练代码

另外，这种训练准确率是沿着训练过程累计的：不同批次使用的参数已经不同；它不完全等于“一轮结束后，用最终参数重新评估整个训练集”的准确率。

7.8 解析评估函数

这是 D2L 3.6 的真实实现：

```
def evaluate_accuracy(net, data_iter):
    if isinstance(net, torch.nn.Module):
        net.eval()                  # 切换评估模式

    metric = Accumulator(2)          # 正确数量、样本总数

    with torch.no_grad():
        for X, y in data_iter:
            metric.add(accuracy(net(X), y), y.numel())

    return metric[0] / metric[1]
```

切换评估模式，不记录求导关系，遍历指定数据集，累计预测正确数和样本数，返回整体准确率。

与训练函数相比，缺少：

```
backward()
zero_grad()
step()
```

原因是：**这里只衡量模型表现，不根据这些标签学习参数。**

如果输入图片是 (32，1，28，28)，模型输出 10 类分数：

```
X:          (32, 1, 28, 28)
net(X):     (32, 10)
argmax 后:  (32,)
y:          (32,)
比较后:     (32,)
正确数:     一个数
```

评估函数还有一个状态变化需要注意：

这个函数结束后，没有自动把模型切回训练模式。

所以接下来继续训练时，需要再次 net.train()。教材上一节的训练函数正是这样做的。

7.9 多轮训练与指标组合

下面节选 D2L train_ch3 的核心循环；假设 animator 已经创建：

```
for epoch in range(num_epochs):
    train_metrics = train_epoch_ch3(net, train_iter, loss, updater)
    test_acc = evaluate_accuracy(net, test_iter)
    animator.add(epoch + 1, train_metrics + (test_acc,))
```

按顺序读：

1. 用训练数据完成一轮参数更新。

2. 用测试数据计算准确率。

3. 把本轮训练指标和测试准确率加入曲线。

循环中的显示轮次与指标组装涉及以下语法：

```
epoch + 1
```

因为循环变量从 0 开始，显示给人看时通常从第 1 轮开始。

```
train_metrics + (test_acc,)
```

这里是**元组拼接**，不是指标数值相加：

```
train_metrics = (训练损失，训练准确率)

拼接后:
(训练损失，训练准确率，测试准确率)
```

(test_acc,) 中的逗号表示单元素元组。

教材每轮展示测试集表现。实际做模型选择时，通常用验证集调整方案，最终测试集留作最后评估；阅读代码时需要区分数据的用途。

7.10 结果展示与检查

常见写法	作用与含义
`d2l.Animator(...)`	创建绘制训练曲线的辅助对象
`animator.add(x, values)`	在对应横坐标加入指标
`d2l.Timer()`	创建计时工具
`timer.start() / timer.stop()`	开始、停止计时
`enumerate(train_iter)`	取数据的同时获得批次编号
`assert 条件，信息`	检查条件，不成立就报错并显示信息

D2L 6.6 的真实打印片段：

```
print(f'loss {train_l:.3f}, train acc {train_acc:.3f}, '
      f'test acc {test_acc:.3f}')
```

显示训练损失、训练准确率、测试准确率，各保留三位小数。

其中：

- `:.3f`：保留三位小数。
- 相邻字符串会拼接，所以虽然写成两行，仍然是一次打印。
- `train_acc = 0.85` 表示 85%；直接按 `.3f` 打印得到 `0.850`。

教材有些 `assert` 用来检查结果是否达到示例预期。**断言没通过，不一定代表代码语法或训练循环写错，也可能是训练结果没有达到设置的阈值。**

阅读训练程序的方法

阅读较长的训练程序时，可先定位以下五部分，再展开局部细节：

要找的位置	常见标记
外层训练进度	`for epoch in ...`
每批参数更新	`backward()`、`step()`
训练与评估切换	`train()`、`eval()`
指标统计	`argmax`、比较、`sum`、`metric.add`
结果展示	`print`、`animator.add`、计时

第八章 设备、存取与综合阅读

本章介绍 GPU、保存与加载，以及 D2L 辅助函数。最后完整阅读一段多层感知机程序，把数据、模型、损失和训练流程连起来。

设备迁移与状态存取不会替代模型训练，但决定了模型能在哪里运行、训练结果怎样保留，以及后续怎样复用。

8.1 计算设备与设备选择

D2L 5.6 GPU经常出现这些写法：

写法	作用与含义
`torch.device('cpu')`	表示 CPU 设备
`torch.device('cuda:0')`	表示第 0 块 CUDA GPU
`torch.cuda.device_count()`	查询当前 PyTorch 能看到的 CUDA GPU 数量
`X.device`	查看张量在哪里
`X.to(device)`	得到放在指定设备上的张量
`net.to(device)`	把模型参数和相关缓冲状态迁到指定设备
`X.cpu()`	得到 CPU 上的张量
`X.cuda()`	迁到 CUDA GPU，需要环境支持

这里的 CUDA 通常对应 NVIDIA GPU。

注意：

```
device = torch.device('cuda:0')
```

只是建立设备描述，并没有把模型或数据迁过去，也不代表这台机器一定支持它。

D2L 用下面的真实函数选择设备：

```
def try_gpu(i=0):
    """如果存在指定GPU，就返回它，否则返回CPU"""
    if torch.cuda.device_count() >= i + 1:
        return torch.device(f'cuda:{i}')
    return torch.device('cpu')
```

默认尝试第 0 块 GPU；如果看不到指定 GPU，就退回 CPU。

据此可写为：

```
device = d2l.try_gpu()
```

`d2l.try_gpu()` 根据可见设备数量选择 GPU 或 CPU。

它不会安装驱动，也不会把 CPU 版本的 PyTorch 自动变成支持 CUDA 的版本。

8.2 模型与数据的设备迁移

下面把 D2L 6.6 中的相关语句摘出来说明，省略训练函数的其他部分：

```
net.to(device)

for X, y in train_iter:
    X, y = X.to(device), y.to(device)
    y_hat = net(X)
    l = loss(y_hat, y)
```

先迁移模型，再逐批迁移输入和标签，在同一设备上预测并计算损失。

张量与模型的 `.to()` 方法对原对象的影响不同。

张量迁移需要使用返回值：

```
X = X.to(device)
```

不要只写：

```
X.to(device)
```

单独调用这一表达式，不会让变量 X 自动绑定到迁移后的张量；若确实发生了迁移，后续仍使用旧 X 会产生设备不匹配。

模型的 `.to()` 会修改模型本身：

```
net.to(device)
```

也可以写：

```
net = net.to(device)
```

模型、输入以及参与损失计算的标签通常需要放在兼容的同一设备上。否则可能出现“发现 cpu 和 cuda 两种设备”之类的报错。

迁移模型通常放在创建优化器之前。D2L 6.6 就采用：

```
建立模型 → 迁移模型 → 创建优化器 → 开始训练
```

绘图或导出结果时，常见以下转换链：

```
# 教学补充
array = X.detach().cpu().numpy()
```

脱离计算图 → 移到 CPU → 转成 NumPy 数组。

这在绘图、结果处理时很常见，不能把它插到还需要反向传播的损失计算路径里。

8.3 张量与容器的存取

D2L 5.5 读写文件的真实片段：

```
x = torch.arange(4)
torch.save(x, 'x-file')
```

它也能保存容器：

```
mydict = {'x': x, 'y': y}
torch.save(mydict, 'mydict')
mydict2 = torch.load('mydict')
```

写法	作用与含义
`torch.save(obj, path)`	把对象写入文件
`torch.load(path)`	从文件恢复对象
`mydict2['x']`	从恢复的字典里取出 x

文件扩展名不是这套机制的决定因素。教材使用没有扩展名的文件，模型参数常见 `.pt`、`.pth`、`.params`。

文件读写还需考虑：

• 相对路径按当前工作目录解析。

• `torch.load()` 返回什么，取决于保存时放了什么；它不会一律返回可直接预测的模型。

8.4 模型状态字典

D2L 保存模型参数的真实写法：

```
torch.save(net.state_dict(), 'mlp.params')
```

可以拆成：

```
# 教学改写
state = net.state_dict()         # 取出状态字典
torch.save(state, 'mlp.params')  # 保存字典
```

字典内容大致像：

```
{
    'hidden.weight': 权重张量,
    'hidden.bias': 偏置张量,
    'output.weight': 权重张量,
    'output.bias': 偏置张量
}
```

有些模型还需要保存不通过梯度学习的数，例如 BatchNorm 记录的均值和方差。这些数也可以包含在状态字典中。

`state_dict` **主要保存“模型当前有哪些数”，不负责把模型类的 Python 定义保存成一份可重建的程序。**

所以恢复时通常先建立结构，再装入这些数。

8.5 重建模型与加载状态

D2L 5.5 的真实片段：

```
clone = MLP()
clone.load_state_dict(torch.load('mlp.params'))
clone.eval()
```

这里的 MLP 必须提前定义好，并与保存时的模型匹配。

语句	作用与含义
`clone = MLP()`	建立同样结构的新模型
`torch.load(...)`	读取保存的状态字典
`clone.load_state_dict(...)`	把字典中的值装入模型
`clone.eval()`	切换到评估模式

新模型构造时通常仍会先产生默认初始化值，随后加载的值覆盖对应参数。

常见失败原因：

情况	可能出现的问题
文件路径不对	找不到文件
层名称改变	参数键缺失或出现多余键
层大小改变	参数形状不匹配
保存于 GPU，加载环境没有对应 GPU	需要指定加载设备映射

`load_state_dict` 的返回值并非模型，以下赋值方式会覆盖原模型变量：

```
# 不要这样理解：load_state_dict 返回的不是模型本身
net = net.load_state_dict(state)
```

正确写法是保留模型变量，单独加载状态：

```
net.load_state_dict(state)
```

8.6 加载设备映射与一致性检查

下面是基于官方接口的**教学补充**，不是 D2L 原句：

```
state = torch.load(
    'mlp.params',
    map_location='cpu',
    weights_only=True
)

clone = MLP()
clone.load_state_dict(state)
clone.eval()
```

参数	意思
`map_location='cpu'`	将加载的张量存储映射到 CPU
`weights_only=True`	限制加载时允许恢复的对象类型，适合本例的模型状态字典

`weights_only=True` 限制加载时允许恢复的对象类型，并不从任意文件中自动筛选权重。文件内容取决于保存时传入的对象。PyTorch 加载接口

加载成功后还可以检查输出。下面是教学补充，假设两个模型已在同一设备，并使用相同结构：

```
net.eval()
clone.eval()

with torch.no_grad():
    before = net(X)
    after = clone(X)

torch.allclose(before, after)
```

`torch.allclose(before, after)` 检查两个输出是否在给定容差内接近。

对于本例这种确定性的模型，同样输入、同样参数和模式应当得到一致或接近的结果。

8.7 训练检查点

只保存：

```
net.state_dict()
```

通常足以配合模型定义恢复预测所需状态。

但断点续训还常需要保存：

```
# 教学补充
torch.save({
    'model': net.state_dict(),
    'optimizer': optimizer.state_dict(),
    'epoch': epoch
}, 'checkpoint.pt')
```

保存模型状态、优化器状态和当前训练轮次。

优化器可能保存动量或历史统计，只恢复权重不等于完整恢复它之前的状态。更完整的复现实验还可能需要调度器、随机状态等。

检查点中不同字段的职责为：

- `model.state_dict()`：模型状态。
- `optimizer.state_dict()`：优化器状态。
- `checkpoint`：通常表示用于恢复进度的检查点。PyTorch 模型保存与恢复教程

恢复训练时，还应把保存的优化器状态装回对应优化器。下面是与上述保存格式配套的教学补充，假设 `net` 与 `optimizer` 已按原结构和算法建立，模型位于 CPU。

```
checkpoint = torch.load('checkpoint.pt', map_location='cpu', weights_only=True)
net.load_state_dict(checkpoint['model'])       # 恢复模型状态
optimizer.load_state_dict(checkpoint['optimizer'])  # 恢复更新器状态
start_epoch = checkpoint['epoch'] + 1          # 假设检查点保存于一轮结束后
net.train()                                    # 恢复训练模式
```

`start_epoch` 还需要用于后续轮次循环。若要逐步复现中断前的轨迹，还需处理随机状态、数据读取进度和其他训练状态；只保存这三个字段并不保证完全相同的后续过程。

8.8 D2L 辅助函数

导入代码：

```
from d2l import torch as d2l
```

它与下面这句不是同一个意思：

```
import torch
```

常见辅助函数及其职责如下：

写法	大致做什么
d2l.synthetic_data(...)	生成教材使用的合成数据
d2l.load_data_fashion_mnist(...)	准备图片数据及训练、测试加载器
d2l.train_ch3(...)	执行第 3 章定义的多轮训练和评估流程
d2l.train_ch6(...)	执行第 6 章定义的训练流程，支持指定设备
d2l.accuracy(...)	在相关分类代码中，计算预测正确数量
d2l.Accumulator(...)	累计统计量
d2l.Animator(...)	更新训练曲线
d2l.try_gpu()	按教材规则选择 GPU 或 CPU

`train_ch3`、`train_ch6` **不是 PyTorch 固定训练接口。它们是教材自己写的函数。**

阅读封装函数时，应检查它接收什么、返回什么，以及是否改变参数、设备或模型模式：

```
接收什么?
返回什么?
内部是否更新参数?
是否处理设备和模式切换?
```

第七章展开的训练封装具有以下调用关系：

```
train_ch3
    ├─ train_epoch_ch3：训练一轮
    ├─ evaluate_accuracy：评估
    └─ Animator：绘制指标
```

查本机安装版本的源码，可以用下面的**教学补充**：

```
import inspect


print(inspect.getsource(d2l.train_ch3))
```

查看源码可以确认安装版本的实际行为，尤其是损失汇总方式、设备迁移与模式切换。函数名只能提供线索，不能代替对实现的检查。

8.9 综合阅读：多层感知机训练

下面合并 D2L 4.3 多层感知机的 PyTorch 代码单元，保留逻辑，添加中文注释：

```
import torch
from torch import nn
from d2l import torch as d2l

# 建立模型
net = nn.Sequential(
    nn.Flatten(),
    nn.Linear(784, 256),
    nn.ReLU(),
    nn.Linear(256, 10)
)

# 定义权重初始化规则
def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight, std=0.01)

# 对模型中的模块应用初始化规则
net.apply(init_weights)

# 设置批大小、学习率、训练轮数
batch_size, lr, num_epochs = 256, 0.1, 10

# 定义逐样本损失和参数更新器
loss = nn.CrossEntropyLoss(reduction='none')
trainer = torch.optim.SGD(net.parameters(), lr=lr)

# 准备数据
train_iter, test_iter = d2l.load_data_fashion_mnist(batch_size)

# 执行训练与评估流程
d2l.train_ch3(
    net, train_iter, test_iter,
    loss, num_epochs, trainer
)
```

按职责划分，这段程序包含以下代码块：

代码块	职责
三条导入	准备张量工具、神经网络工具、教材辅助函数
`nn.Sequential(...)`	图片展开后，通过两个全连接层和一个激活层，输出 10 类分数
`init_weights` 与 `apply`	重新设置全连接层的权重初值
三个超参数	每批 256 条，学习率 0.1，训练 10 轮
损失与优化器	逐样本计算交叉熵，用 SGD 调整模型参数
数据加载	取得训练和测试数据加载器
`train_ch3`	循环训练、评估、统计并绘制指标

数据形状依次变化如下：

```
图片:         (B, 1, 28, 28)
展开:         (B, 784)
隐藏层:       (B, 256)
激活:         (B, 256)
输出分数:     (B, 10)
标签:         (B,)
逐样本损失:   (B,)
平均损失:     标量
```

`B` 表示当前批的实际样本数，最后一批可能不足 256。

这段程序没有显式执行 GPU 迁移。硬件具备 GPU、安装了 PyTorch，都不意味着模型和数据已经在 GPU 上计算。

如果只加一句 `net.to(device)`，却没让训练函数同时迁移数据，可能导致设备不匹配。使用某个训练封装前，要先确认它是否负责这件事。